since the fourth bit hadn’t been set yet (`B[0]==0`), a lookup would have returned `false`.

This was of course a simplistic example intentionally crafted to prove a point: false positives are possible and they are more likely to happen if we don't choose the parameters of our Bloom filter carefully. In the section 4.10, we'll learn how to tune these parameters, based on the number of elements we anticipate we will store, and on the precision we need.

4.8.3 Bloom Filters as Randomized Algorithms

Appendix F introduces the taxonomy of randomized algorithms, and in particular the dichotomy between *Las Vegas* and *Monte Carlo* algorithms.

If you don’t have a clear idea of what’s the difference between these two classes or what are randomized algorithms, this would be a good time to check appendix F.

Once those definitions are clear to you, it should not be hard to make an educated guess: which category do our Bloom filters belong to?

As you might have already figured out, Bloom filters are an example of a Monte Carlo data structure. Method `contains`, the algorithm that checks if a key is stored in a Bloom filter, in particular, is a *false-biased* algorithm: in fact, it might return `true` for some keys never added to the filter, but it always correctly returns `true` when a key was previously